

AWS Self-Learning and Hands-On Implementation Activity

Name: Md Zeeshan Uddin

Activity: AWS Self-Learning and Hands-On Implementation

Date: 30 September 2026

Region: US East (N. Virginia) – us-east-1

1. Objective

The objective of this activity was to understand and perform hands-on implementation of commonly used AWS services.

The following services were studied and implemented:

1. Amazon SQS
2. Amazon MQ
3. AWS Secrets Manager

The activity focused on understanding the **concept, use case, basic configuration, testing, and observations** of each service.

1. Amazon SQS

What is Amazon SQS?

Amazon SQS (Simple Queue Service) is a managed message queue service. It allows applications to communicate asynchronously.

Use Case

Example:

Order Application

|

v

SQS Queue

|

v

Processing Application

The producer sends a message to the queue, and the consumer receives and processes it.

Architecture

Producer

|

| Send Message

v

SQS Queue

|

| Receive Message

v

Consumer

Implementation

Created a Standard SQS queue:
zeeshan-order-queue

A test message was sent:

Order 101

The message was then received using:

Send and receive messages → Poll for messages

After successful processing, the message was deleted.

Message Flow

Send



SQS Queue



Receive

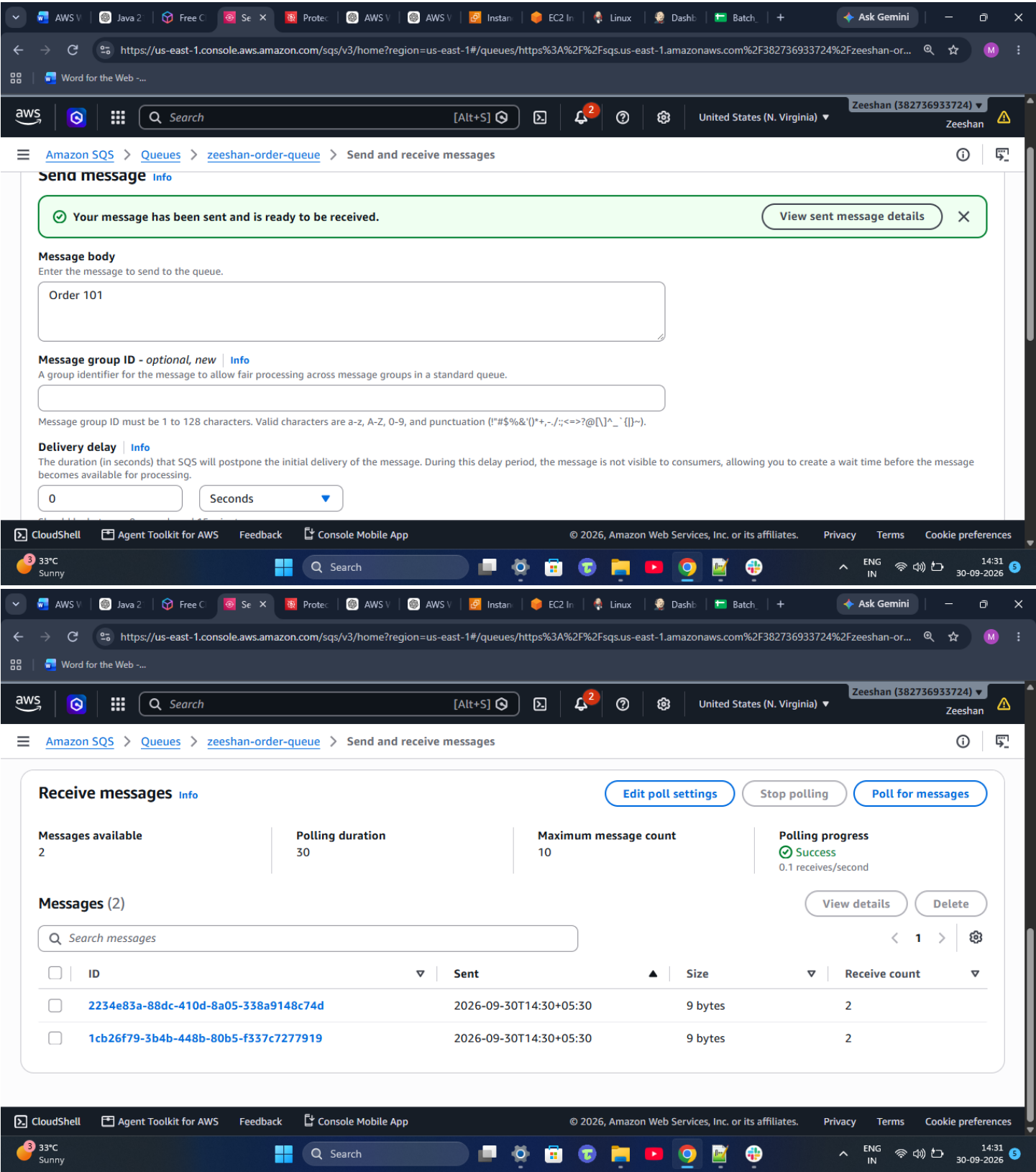


Process



Delete

Screenshot



Observation

SQS helps decouple applications. The producer does not need to directly communicate with the consumer.

2. Amazon MQ

What is Amazon MQ?

Amazon MQ is a managed message broker service.

It supports messaging technologies such as:

- RabbitMQ
- ActiveMQ

Use Case

Amazon MQ can be useful when an existing application already uses a traditional message broker such as RabbitMQ or ActiveMQ.

Architecture

Application

|

v

Amazon MQ

|

v

RabbitMQ Broker

|

+---- Producer

|

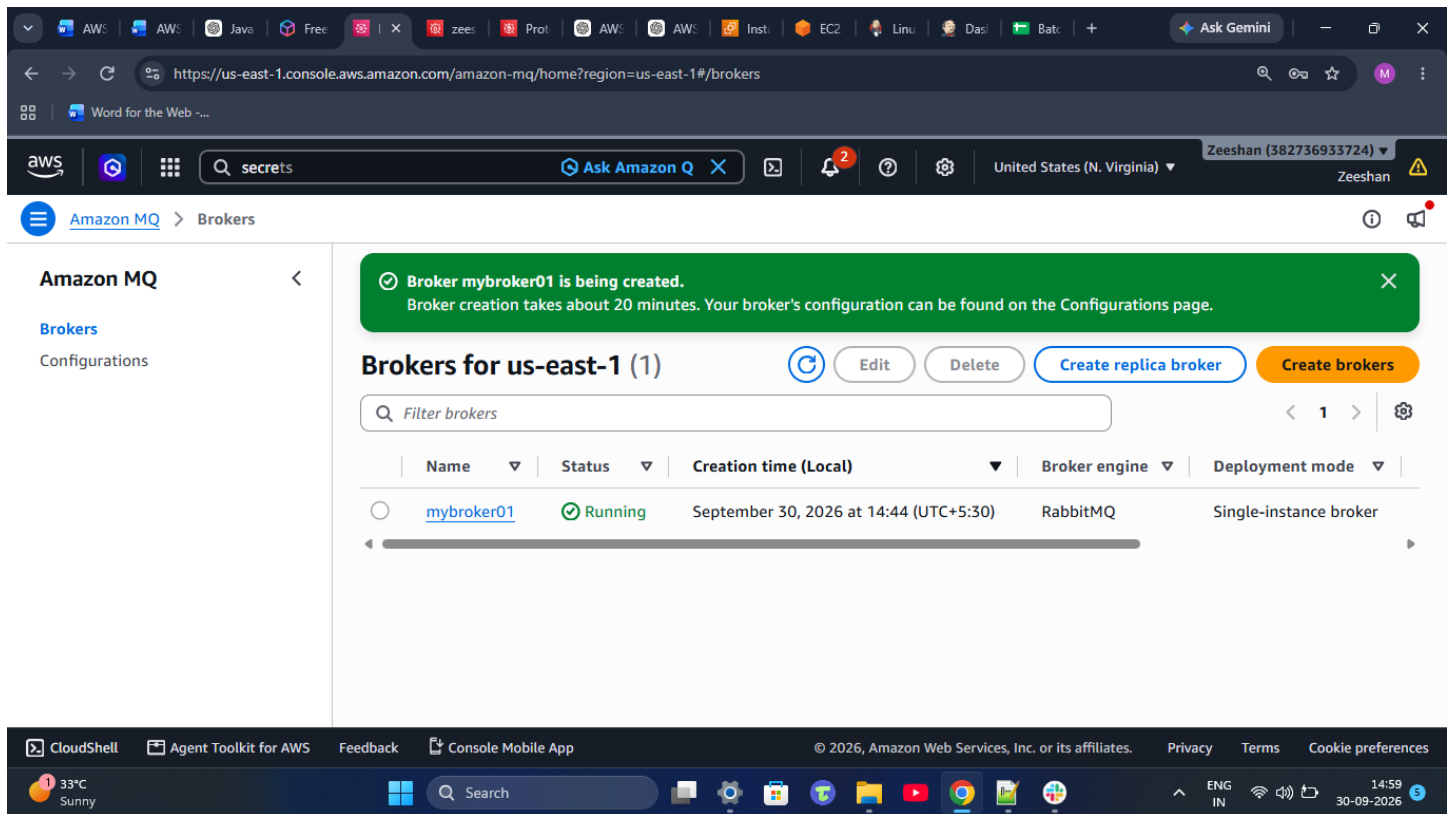
+---- Consumer

Implementation

For the lab:

1. Opened **Amazon MQ**.
2. Selected **RabbitMQ** as the broker engine.
3. Configured broker credentials.
4. Created the broker.
5. Checked the broker configuration and status.

Screenshot



Observation

Amazon MQ provides a managed environment for traditional message brokers, reducing the need to manually manage the broker infrastructure.

Cost Observation

Amazon MQ can generate charges while the broker is running. The broker should be deleted after completing the learning activity if it is no longer required.

3. AWS Secrets Manager

What is Secrets Manager?

AWS Secrets Manager is used to securely store and retrieve sensitive information such as:

- Database passwords
- API keys
- Application credentials
- Other secrets

Use Case

Instead of storing passwords directly in application code:

username = admin

password = password123

the application retrieves the credentials from Secrets Manager.

Architecture

Application

|
v

Secrets Manager

|

v

Username / Password

Implementation

Created a secret:

zeeshan/dev/app-secret

Example values:

username = appuser

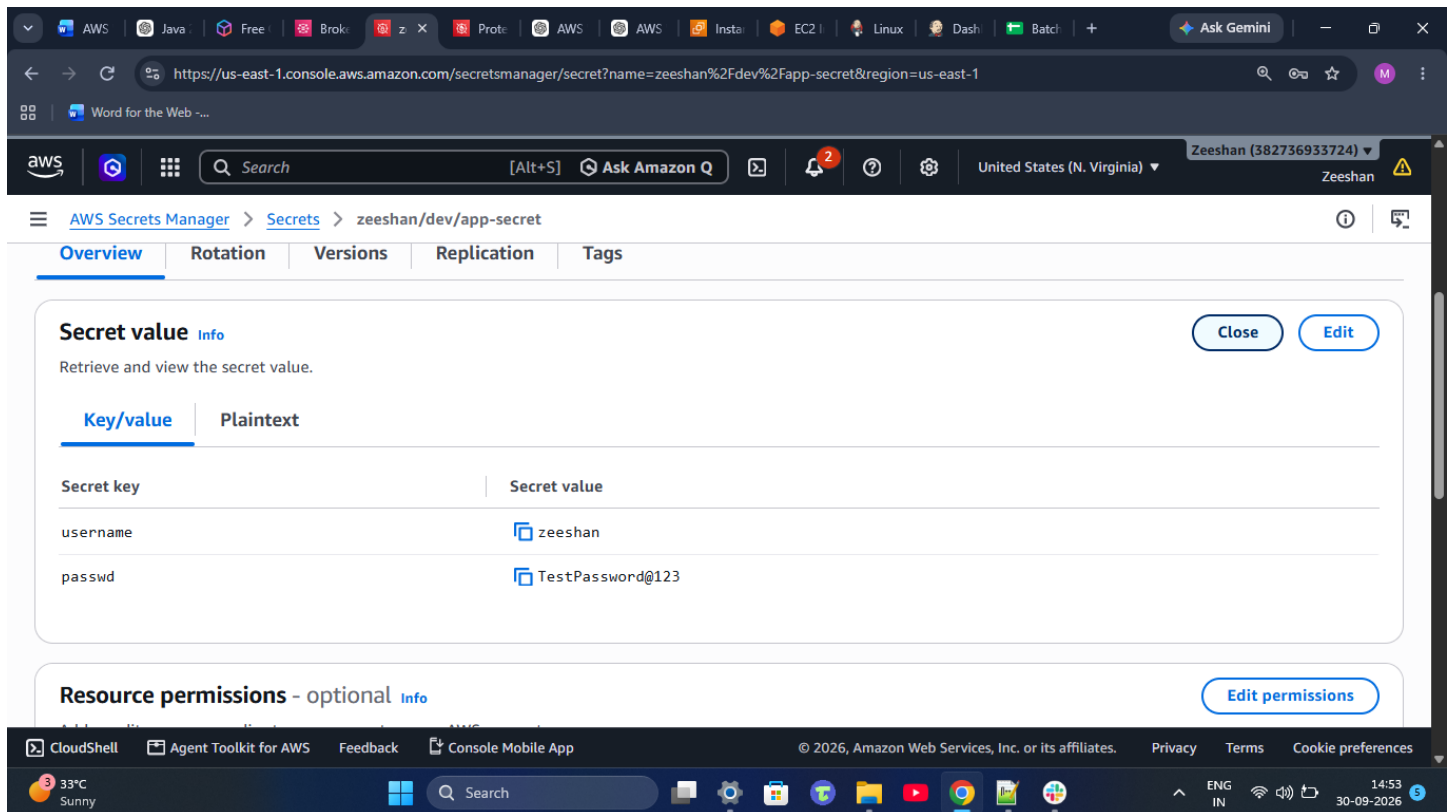
password = TestPassword@123

The secret was stored and then retrieved from the Secrets Manager console.

CLI Test

```
aws secretsmanager get-secret-value \
--secret-id "zeeshan/dev/app-secret"
```

Screenshot



Observation

Secrets Manager provides a secure way to store sensitive application information and allows access to be controlled through IAM permissions.

Key Learning.

- How SQS sends, receives, and deletes messages.
- Basic concept and use case of Amazon MQ.
- Difference between SQS and Amazon MQ.
- How Secrets Manager stores sensitive credentials.

- How IAM can control application access to AWS services.
- How AWS services can be combined in a DevOps architecture.

Conclusion

The hands-on activity provided practical knowledge of AWS security, configuration management, messaging, and secret management services.

The implementation helped understand how these services can be used in real-world DevOps environments for **application protection, configuration management, asynchronous communication, message brokering, and secure credential management.**